PRACTICAL - 11

Aim: Save/load serialization (Student.java + Driver.java)

Code Screenshot:

Visual Studio Code - Student.java

Student.java

```java
import java.io.Serializable;

public class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String name;
    private transient String password;

    public Student(int id, String name, String password) {
        this.id = id;
        this.name = name;
        this.password = password;
    }

    public String toString() {
        return id + " - " + name + " | password=" + password;
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.io.*;
import java.nio.file.*;

public class Driver {
    public static void main(String[] args) throws Exception {
        Student[] students = {
            new Student(101, "Aarav", "secret101"),
            new Student(102, "Meera", "secret102")
        };
        Path file = Path.of("students.dat");
        try (ObjectOutputStream out = new ObjectOutputStream(Files.newOutputStream(file))) {
            out.writeObject(students);
        }
        Student[] loaded;
        try (ObjectInputStream in = new ObjectInputStream(Files.newInputStream(file))) {
            loaded = (Student[]) in.readObject();
        }
        System.out.println("Saved students: " + students.length);
        for (Student student : loaded) System.out.println("Loaded: " + student);
        Files.deleteIfExists(file);
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered Java object persistence using java.io.Serializable and serialVersionUID.
- Managed binary serialization streams with ObjectOutputStream and ObjectInputStream.
- Protected sensitive credentials using the transient modifier.
- Ensured robust I/O leak prevention through try-with-resources.
- Understood JVM byte-stream reconstitution mechanics.

PRACTICAL - 11

Aim: Log analyzer (LogAnalyzer.java + Driver.java)

Code Screenshot:

Visual Studio Code - LogAnalyzer.java

LogAnalyzer.java

```java
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.Stream;

public class LogAnalyzer {
    public static void analyze(Path[] files, String keyword) throws IOException {
        long totalLines = 0;
        long matches = 0;
        for (Path file : files) {
            AtomicInteger lineCount = new AtomicInteger();
            AtomicInteger matchCount = new AtomicInteger();
            try (Stream<String> lines = Files.lines(file)) {
                lines.forEach(line -> {
                    lineCount.incrementAndGet();
                    if (line.contains(keyword)) matchCount.incrementAndGet();
                });
            }
            BasicFileAttributes attr = Files.readAttributes(file, BasicFileAttributes.class);
            totalLines += lineCount.get();
            matches += matchCount.get();
            System.out.println(file.getFileName() + " | Size: " + attr.size() + " bytes");
        }
        System.out.println("Total Lines: " + totalLines);
        System.out.println("Matches for '" + keyword + "': " + matches);
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.nio.file.*;
import java.util.List;

public class Driver {
    public static void main(String[] args) throws Exception {
        Path folder = Path.of("logs");
        Files.createDirectories(folder);
        Path first = folder.resolve("app1.log");
        Path second = folder.resolve("app2.log");
        Files.write(first, List.of("INFO Application started", "ERROR Database timeout", "INFO Retrying connection"));
        Files.write(second, List.of("INFO User login", "WARN Low memory", "ERROR Connection lost"));
        LogAnalyzer.analyze(new Path[]{first, second}, "ERROR");
        Files.deleteIfExists(first);
        Files.deleteIfExists(second);
        Files.deleteIfExists(folder);
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered modern Java NIO.2 file handling with Path, Paths, and Files.
- Inspected OS file attributes including file size using Files.size().
- Streamed and filtered text lines efficiently with Java Collections and Streams.
- Implemented high-performance file content search pipelines.
- Applied safe temporary resource creation and deletion operations.

PRACTICAL - 11

Aim: NIO directory tree report (For Advanced Learners) (DirectoryReport.java + Driver.java)

Code Screenshot:

Visual Studio Code - DirectoryReport.java

DirectoryReport.java

```java
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;

public class DirectoryReport {
    public static int create(Path root, Path report) throws IOException {
        List<String> rows = new ArrayList<>();
        try (var paths = Files.walk(root)) {
            for (Path path : paths.filter(Files::isRegularFile).toList()) {
                BasicFileAttributes a = Files.readAttributes(path, BasicFileAttributes.class);
                String modified = a.lastModifiedTime().toInstant()
                        .truncatedTo(ChronoUnit.SECONDS).toString();
                rows.add(path.getFileName() + " | " + a.size() + " bytes | " + modified);
            }
        }
        Files.write(report, rows);
        return rows.size();
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.nio.file.*;
import java.util.List;

public class Driver {
    public static void main(String[] args) throws Exception {
        Path root = Path.of("sample_tree");
        Files.createDirectories(root.resolve("docs"));
        Files.createDirectories(root.resolve("logs"));
        Files.writeString(root.resolve("docs/readme.txt"), "OOP practical report\n");
        Files.write(root.resolve("logs/app.log"), List.of("INFO started", "ERROR failed"));
        Path report = Path.of("file_report.txt");
        int count = DirectoryReport.create(root, report);
        System.out.println("Files found: " + count);
        System.out.println("Report: " + report);
        for (String line : Files.readAllLines(report)) System.out.println(line);
    }
}
```

Output Screenshot:



Learning Outcome:

- Traversed nested directories using the Java NIO Files API.
- Read file names, sizes, and last-modified times with BasicFileAttributes.
- Collected file metadata into a structured report.
- Wrote the generated directory summary to an output file.
- Validated file reporting across multiple folders and file types.